

Group Project: Image Guided Robotic Needle Placement

Mohamad Nassif, Yannic Hillers, Elisa Janowski, Clara Schumacher, Sören Anspach
Institute of Medical Technology and Intelligent Systems
Hamburg University of Technology (TUHH)
Hamburg, Germany

Abstract—Robot-assisted needle insertion requires accurate perception of the operating environment, localization of an insertion target, generation of a collision-free approach, and coordinated execution by the manipulator. This work presents an autonomous needle-insertion system developed using a seven-degree-of-freedom Franka Emika Panda robot equipped with an eye-in-hand Azure Kinect RGB-D camera. The complete workflow is implemented as a modular ROS 2 system integrating camera and hand-eye calibration, automated multi-view scanning, point-cloud reconstruction, model registration, target localization, geometric entry-point planning, inverse kinematics, insertion-path generation, and time-parameterized joint trajectory execution. A high-resolution STL model containing a predefined target is registered to a reconstructed skeleton phantom, allowing semantic target information to be transferred into the robot base frame. Candidate needle paths are subsequently evaluated according to geometric clearance and insertion constraints before robot feasibility is considered. The resulting Cartesian insertion path is transformed into joint-space waypoints and executed using quintic trajectories subject to velocity, acceleration, and jerk limits. The developed system demonstrates an integrated perception-to-action architecture for autonomous model-based needle insertion.

Index Terms—robot-assisted needle insertion, RGB-D perception, point-cloud registration, hand-eye calibration, inverse kinematics, trajectory planning, ROS 2

I. INTRODUCTION

Needle insertion is used in minimally invasive procedures such as biopsy, localized drug delivery, and diagnostic interventions. Reliable insertion requires accurate target localization, a suitable approach direction, and safe robot motion. Robotic assistance can improve repeatability by combining sensing, geometric planning, and automated execution. This work presents an autonomous needle-insertion pipeline using a Franka Emika Panda manipulator with an eye-in-hand Azure Kinect RGB-D camera. The skeleton phantom is scanned from multiple viewpoints, and the acquired point clouds are transformed into the robot base frame and combined into a common three-dimensional reconstruction. A high-resolution STL model containing a predefined target is then registered to the measured point cloud, allowing the target location to be transferred to the physical scene. Based on the registered model, geometrically feasible needle-entry candidates are generated and evaluated. The selected entry and target are converted into a Cartesian insertion path, while inverse kinematics, joint-limit checks, and lightweight collision checks are used to determine robot feasibility. Finally, smooth joint trajectories

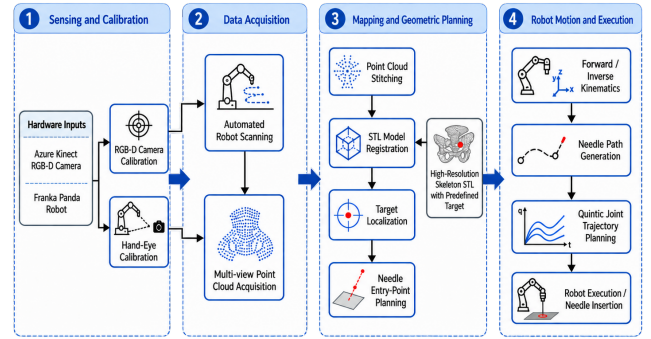


Fig. 1. Overview of the autonomous needle-insertion pipeline. Image created with AI

are generated while respecting velocity, acceleration, and jerk constraints. The resulting ROS 2 system integrates calibration, perception, registration, geometric planning, kinematics, and trajectory generation into a complete perception-to-action workflow for autonomous needle insertion.

II. SYSTEM ARCHITECTURE

The system is implemented in ROS 2 Humble and divided into functional packages for calibration, hand-eye estimation, scanning, mapping, kinematics, needle-path generation, trajectory planning, and supporting tools. The complete processing chain is

calibration → hand-eye estimation → scanning
→ 3D reconstruction → registration → target planning (1)
→ IK → trajectory generation → insertion.

The Franka base frame, `panda_link0`, is used as the principal reference frame. The transformation chain connecting the robot and sensor is

$$\mathcal{F}_B \rightarrow \mathcal{F}_{EE} \rightarrow \mathcal{F}_{RGB} \rightarrow \mathcal{F}_D, \quad (2)$$

where B , EE , RGB , and D denote the robot base, end-effector, RGB camera, and depth camera frames, respectively.

The perception subsystem produces entry and target poses directly in the base coordinate frame. These poses form a clean interface between model-based perception and robot-specific planning. The architecture consequently separates

geometric feasibility from manipulator feasibility: the mapping subsystem determines anatomically admissible paths, while the kinematic and trajectory subsystems determine whether those paths can be executed by the robot.

A. Team Contributions

The development was divided among the main functional components of the system. Mohamad Nassif was responsible for RGB-D camera calibration, multi-view point-cloud reconstruction, STL registration, target localization, and needle-entry planning. Sören Anspach developed the hand-eye calibration component, while Yannic Hillers was responsible for automated robot scanning, and for setting up the general repository structure, including the ROS2 workspace and Git version control. Elisa Janowski developed the needle insertion-path generation in cartesian space with transformation into joint coordinates. Clara Schumacher was responsible for joint-trajectory planning and the movement execution pipeline. Kinematics design, integration, debugging, and validation of the complete perception-to-action pipeline were carried out collaboratively by the team.

III. IMPLEMENTATION

A. RGB-D Camera Calibration

Accurate RGB-D reconstruction requires reliable intrinsic calibration of both camera sensors and estimation of the rigid transformation between them. A custom Azure Kinect calibration module was implemented using synchronized RGB and infrared images of a planar checkerboard. The RGB and depth-camera intrinsics were calibrated independently using 25 checkerboard observations, following the planar calibration method described by Zhang [1]. Stereo calibration was then performed with the intrinsic parameters fixed to estimate the relative rotation and translation between the RGB and depth sensors. The resulting calibration achieved root-mean-square reprojection errors of approximately 0.153 pixels for the RGB camera, 0.530 pixels for the depth/IR camera, and 0.470 pixels for the stereo calibration. The calculated intrinsic and extrinsic parameters were subsequently published through ROS 2 camera information and static frame transformations for use by the reconstruction pipeline.

B. Hand-Eye Calibration and Automated Scanning

To determine the position of the phantom with respect to the robot base frame, multiple point clouds are acquired in the camera coordinate frame. Consequently, the pose of the camera with respect to the robot base frame must be determined. This can be achieved by using the end-effector pose calculated by the forward kinematics and by using the transformation between the end-effector and the camera frame. This transformation can be estimated using the eye-in-hand hand-eye calibration. The calibration is performed by moving the robot to multiple different poses and taking snapshots when reaching the target positions. A snapshot refers to a synchronized acquisition consisting of an RGB image and the corresponding end-effector pose. The data contained in

the snapshots are used to calculate the relative motions. The problem can be expressed using the classical relation

$$\mathbf{A}_i \mathbf{X} = \mathbf{X} \mathbf{B}_i, \quad (3)$$

where $\mathbf{A}_i$ describes the relative motions of the robot and $\mathbf{B}_i$ describes the motions of the checkerboard relative to the camera. $\mathbf{X}$ represents the desired transformation from the camera to the end-effector.

The checkerboard used in this project consists of 8 rows and 10 columns of squares, each with a side length of 25 mm. The checkerboard is detected in the RGB images using OpenCV. For the hand-eye calibration, the robot can be moved manually or automatically using the scanning subsystem, and at least 30 snapshots are acquired. Snapshots are only accepted when the corners of the checkerboard can be detected in the image and when the forward kinematics node is able to provide the pose of the end-effector. The transformation is estimated using the Tsai-Lenz hand-eye calibration method in OpenCV.

The scanning subsystem coordinates robot motion and data acquisition. It operates in two modes. During hand-eye calibration, camera poses are varied within a predefined workspace while maintaining visibility of the checkerboard. An image is accepted only after the robot has completed its motion and the calibration target remains visible.

During model acquisition, predefined seven-joint scan configurations are loaded from a configuration file. A state machine commands each pose, verifies that robot feedback has reached the goal, and waits for confirmation that the point cloud was captured before moving to the next configuration. Motion-start detection, command retries, joint-feedback freshness checks, and capture timeouts are included to prevent stale completion messages from advancing the scanning sequence incorrectly.

C. Multi-View Point-Cloud Reconstruction

A single depth image contains only the surface visible from one camera viewpoint. Multiple point clouds are therefore accumulated into a common reconstruction of the skeleton phantom.

Because the camera pose is determined from robot kinematics and the calibrated transform chain, visual odometry or SLAM is not required. For every cloud, the transform from its sensor frame to the robot base frame is queried at the point-cloud timestamp. Each measured point is transformed according to

$${}^B \mathbf{p} = {}^B \mathbf{R}_C {}^C \mathbf{p} + {}^B \mathbf{t}_C. \quad (4)$$

Timestamp-consistent transformations are particularly important in the eye-in-hand configuration because associating a cloud with a robot pose from a different instant directly introduces spatial misalignment.

The stitcher accepts measurements only when the camera is considered settled. In the implemented configuration, the sensor pose must remain within approximately 1 mm translation and 0.005 rad rotation for one second. Points outside the



Fig. 2. Multi-view reconstruction and registration of the high-resolution reference model.

useful camera depth interval of approximately 0.25–0.9 m are removed before transformation.

Voxel-grid downsampling reduces redundancy and sensor noise, while radius-based neighbour filtering removes isolated points. The accumulated cloud additionally maintains observation support for occupied voxels, allowing isolated geometry observed in only one measurement to be rejected. An optional cloud-level overlap test can reject entire views that are inconsistent with the existing reconstruction.

The resulting point cloud is expressed in `panda_link0` and stored as a PLY representation for subsequent model registration.

D. Reference-Model Registration

The measured point cloud represents the physical skeleton but does not contain the semantic target required for insertion. The high-resolution reference STL is therefore registered to the measured reconstruction.

Surface points are sampled from the STL, converted from millimetres to metres, and downsampled. Registration consists of a coarse principal-component initialization followed by trimmed point-to-point Iterative Closest Point (ICP) refinement.

For a centered point set $\mathbf{x}_i$, PCA is computed from

$$\mathbf{C} = \frac{1}{N-1} \sum_{i=1}^N \mathbf{x}_i \mathbf{x}_i^T. \quad (5)$$

The eigenvectors of $\mathbf{C}$ describe the principal geometric axes. Since PCA eigenvector signs and axis assignments are ambiguous, the implementation evaluates axis permutations and sign combinations. Candidate transformations corresponding to reflections, for which $\det(\mathbf{R}) < 0$, are discarded. Remaining candidates are ranked using trimmed nearest-neighbour error.

The best initialization is refined with ICP [2]. Given corresponding scan and model points $\mathbf{p}_i$ and $\mathbf{q}_i$, the rigid transformation minimizes

$$E(\mathbf{R}, \mathbf{t}) = \sum_i \|\mathbf{R}\mathbf{p}_i + \mathbf{t} - \mathbf{q}_i\|^2. \quad (6)$$

Nearest neighbours are obtained using a KD-tree. Large residuals are removed and only the best correspondence frac-

tion is retained, reducing the influence of incomplete geometry and remaining outliers. The optimal incremental rigid transform is obtained using singular-value decomposition of the cross-covariance matrix [3].

The resulting homogeneous transformation

$${}^B\mathbf{T}_M = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \quad (7)$$

maps arbitrary points from the reference-model frame M into the physical robot base frame B .

E. Target Localization and Needle-Entry Planning

Rather than attempting to detect the small target directly in the noisy depth reconstruction, target localization is performed on the high-resolution STL. The model is voxelized and separated into connected components. Candidate components are filtered according to physical dimensions and compactness,

$$c = \frac{d_{\min}}{d_{\max}}, \quad (8)$$

where $d_{\min}$ and $d_{\max}$ are the minimum and maximum component extents. Compact approximately spherical components are preferred.

The selected target center is transformed into the robot base frame using

$$\begin{bmatrix} {}^B\mathbf{p}_t \\ 1 \end{bmatrix} = {}^B\mathbf{T}_M \begin{bmatrix} {}^M\mathbf{p}_t \\ 1 \end{bmatrix}. \quad (9)$$

The transformed target is validated against the measured point cloud using its position relative to the scan bounds, the nearest measured surface distance, and local point support.

Needle-entry candidates are then sampled from a plane outside the registered skeleton. For a candidate entry $\mathbf{p}_e$ and target $\mathbf{p}_t$, the insertion direction is

$$\mathbf{a} = \frac{\mathbf{p}_t - \mathbf{p}_e}{\|\mathbf{p}_t - \mathbf{p}_e\|}. \quad (10)$$

Candidates exceeding the available needle length or violating the allowed insertion-angle interval are rejected. Mesh intersection tests prevent a candidate line from crossing anatomical geometry before reaching the target. Clearance is additionally evaluated at points sampled along the complete insertion segment.

The implemented configuration uses a 160 mm needle, a nominal 5 mm needle diameter, insertion elevations between 10° and 50°, a hard model-clearance requirement of 5 mm, and a preferred clearance of 10 mm. Multiple valid candidates are retained instead of only the highest-scoring one, allowing downstream robot-specific feasibility tests to consider alternatives.

F. Robot Kinematics and Needle Path

Forward kinematics are implemented using a modified Denavit–Hartenberg representation of the seven-joint Panda manipulator [6]. Each joint transformation is multiplied sequentially to obtain the end-effector pose,

$${}^0\mathbf{T}_{EE} = \prod_{i=1}^n {}^{i-1}\mathbf{T}_i. \quad (11)$$

The inverse-kinematics solver uses a numerical Jacobian and damped least-squares update. For a six-dimensional Cartesian pose error $\mathbf{e}$ and manipulator Jacobian $\mathbf{J}$, the update is

$$\Delta \mathbf{q} = \mathbf{J}^T (\mathbf{J}\mathbf{J}^T + \lambda^2 \mathbf{I})^{-1} \mathbf{e}, \quad (12)$$

where λ is the damping coefficient [4]. Per-iteration joint updates are bounded and the resulting joint states are clipped to physical joint limits.

The main execution node (`insertion_path_node`) operates within a talker/listener architecture. It subscribes to `/joint_states` to obtain the initial robot configuration $\mathbf{q}_{\text{start}}$ and receives target and entry locations ($\mathbf{p}_{\text{target}}, \mathbf{p}_{\text{entry}} \in \mathbb{R}^3$) via the `/needle_poses` topic as a `PoseArray` or parses them from vision report files.

To align the local tool axis with the insertion trajectory, the unit direction vector is computed as

$$\mathbf{z} = \frac{\mathbf{d}}{\|\mathbf{d}\|}, \quad \text{with } \mathbf{d} = \mathbf{p}_{\text{target}} - \mathbf{p}_{\text{entry}}. \quad (13)$$

Since a single direction vector leaves rotation around the needle axis underdetermined, a stable orthonormal rotation matrix $\mathbf{R} = [\mathbf{x} \ \mathbf{y} \ \mathbf{z}] \in SO(3)$ is constructed using a cascading cross-product scheme. A reference vector $\mathbf{up} = [0, 0, 1]^T$ is selected (switched to $[0, 1, 0]^T$ if $|\mathbf{up} \cdot \mathbf{z}| > 0.9$). The normalized transverse axis $\mathbf{x}$ and the orthogonal axis $\mathbf{y}$ are derived as

$$\mathbf{x} = \frac{\mathbf{up} \times \mathbf{z}}{\|\mathbf{up} \times \mathbf{z}\|}, \quad \mathbf{y} = \mathbf{z} \times \mathbf{x}. \quad (14)$$

The matrix $\mathbf{R}$ is subsequently converted into an orientation quaternion $q = (q_x, q_y, q_z, q_w)$.

Directly solving IK for the needle tip poses would cause positioning errors, as the kinematics solver targets the robot end-effector flange. The kinematic chain is therefore extended by accounting for a 167 mm needle offset ($z_{\text{offset}} = 0.167$ m). For every tip pose $\mathbf{p}_{\text{tip}}$, the required flange position $\mathbf{p}_{\text{flange}}$ is shifted along the insertion direction:

$$\mathbf{p}_{\text{flange}} = \mathbf{p}_{\text{tip}} - \mathbf{z} \cdot z_{\text{offset}}. \quad (15)$$

For safety, a pre-entry pose is established 50 mm (0.05 m) behind the entry point,

$$\mathbf{p}_{\text{pre}} = \mathbf{p}_{\text{entry}} - \mathbf{z} \cdot (0.05 + z_{\text{offset}}), \quad (16)$$

ensuring full orientation alignment before skin contact. The insertion path is linearly interpolated from entry to target at approximately 10 mm Cartesian spacing.

Each Cartesian waypoint is solved sequentially using IK, using the previous solution as the seed for the next waypoint

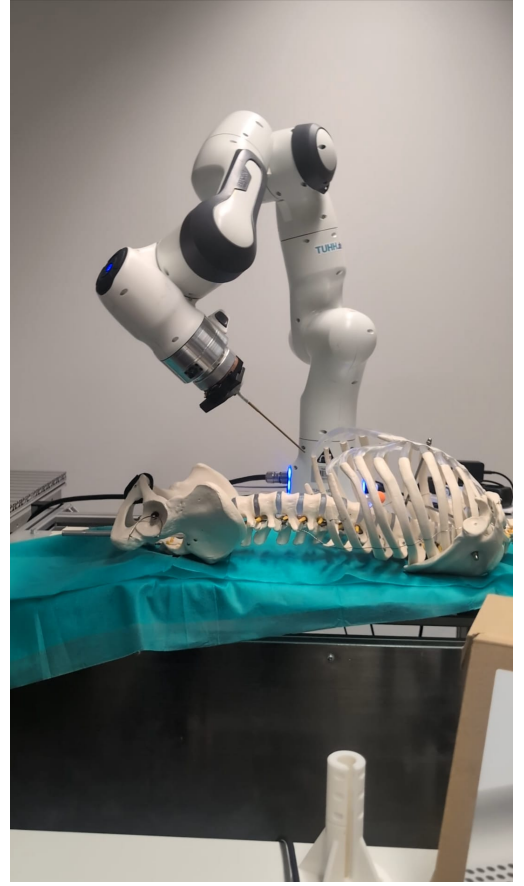


Fig. 3. Franka Emika Panda robot positioned for needle insertion into the skeleton phantom.

to encourage continuous joint motion. For the initial pre-entry pose, several alternative IK seeds are evaluated. Candidate configurations are rejected when they approach joint limits excessively (margin < 0.05 rad), produce excessive joint-space jumps ($\Delta q > 0.8$ rad), or fail the lightweight collision checks (evaluating self-collisions, table boundary at $z_{\text{table}} = -0.05$ m, and forbidden sphere regions). The best safe pre-entry solution is selected according to distance from the current configuration together with penalties for poor joint-limit margin and large elbow or wrist displacement.

The resulting joint path is formatted as a `trajectory_msgs/JointTrajectory` message with predefined time intervals (0.5 s per waypoint) and published on the `/needle_path` topic. This stage converts the geometrically feasible insertion line into a robot-feasible sequence of joint configurations, serving as the direct input for downstream quintic trajectory generation.

G. Trajectory Generation and Execution

Trajectory generation ensures smooth motion across all robot joints by parameterizing the received joint-coordinate waypoints over time using fifth-order (quintic) polynomials. Quintic trajectories ensure C^2 continuity, which is essential for smooth manipulator motion.

The trajectory generator supports two different path execution modes: point-to-point motion and needle insertion. Point-to-point motion is utilized during automated hand-eye calibration and robotic scanning. The needle insertion pipeline divides the trajectory into three phases:

- 1) Motion from the current state to a pre-entry pose,
- 2) Movement from the pre-entry pose to the entry pose on the chest phantom,
- 3) Insertion from the entry pose to the target pose.

The approach and insertion phases can be transmitted independently, ensuring that insertion does not begin until the completion of the approach is verified. Furthermore, the duration of each phase is set independently, allowing for a fast approach and slower high-precision movement during insertion.

The duration of movement between consecutive waypoints is calculated per joint using the absolute distance between the joints $|\Delta q_i|$. The segment duration for each constraint is calculated using the formulation for fifth-order polynomials from [7]:

$$t_{v,i} = 1.875 \cdot \frac{|\Delta q_i|}{\dot{q}_{\text{limit},i}} \quad (17)$$

$$t_{a,i} = \sqrt{5.7735 \cdot \frac{|\Delta q_i|}{\ddot{q}_{\text{limit},i}}} \quad (18)$$

$$t_{j,i} = \sqrt[3]{60 \cdot \frac{|\Delta q_i|}{\dddot{q}_{\text{limit},i}}} \quad (19)$$

where $\dot{q}_{\text{limit},i}$, $\ddot{q}_{\text{limit},i}$, and $\dddot{q}_{\text{limit},i}$ represent the velocity, acceleration, and jerk limits specified in the robot's manual, scaled down to 3 % by a safety factor. The longest time for each joint is chosen. To synchronize movement across all joints, the segment duration T is governed by the most constrained joint.

To enforce C^2 continuity along the trajectory, a fifth-order polynomial is assigned to each joint i :

$$q_i(t) = a_0 + a_1t + a_2t^2 + a_3t^3 + a_4t^4 + a_5t^5. \quad (20)$$

Its time derivatives are given by:

$$\dot{q}_i(t) = a_1 + 2a_2t + 3a_3t^2 + 4a_4t^3 + 5a_5t^4 \quad (21)$$

$$\ddot{q}_i(t) = 2a_2 + 6a_3t + 12a_4t^2 + 20a_5t^3. \quad (22)$$

Six boundary conditions specifying initial and final positions, velocities and accelerations define the six unknown coefficients (a_0 through a_5). Evaluating these conditions over segment duration T yields the linear system:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 \\ 1 & T & T^2 & T^3 & T^4 & T^5 \\ 0 & 1 & 2T & 3T^2 & 4T^3 & 5T^4 \\ 0 & 0 & 2 & 6T & 12T^2 & 20T^3 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ a_4 \\ a_5 \end{bmatrix} = \begin{bmatrix} q(0) \\ \dot{q}(0) \\ \ddot{q}(0) \\ q(T) \\ \dot{q}(T) \\ \ddot{q}(T) \end{bmatrix}. \quad (23)$$

Standard trajectory generation assigns zero boundary conditions ($\dot{q} = \ddot{q} = 0$) at all segment endpoints. While not impacting path accuracy, this forces the robot to come to a

TABLE I
REPRESENTATIVE PIPELINE RESULTS

Quantity	Result
RGB calibration RMS	0.153 px
Depth/IR calibration RMS	0.530 px
Stereo calibration RMS	0.470 px
Target nearest scan distance	4.086 mm
Target support points (50 mm radius)	212
Valid insertion candidates	286
Selected path length	159.4 mm
Selected path elevation	40.1°
Minimum path clearance	5.775 mm
Mean path clearance	29.185 mm

complete stop at intermediate waypoints along the insertion path. To evaluate continuous motion across multiple waypoints, average velocities were calculated and assigned to intermediate waypoints, using zero-velocity conditions only for the initial and final trajectory endpoints. However, this would lead to different coefficients for execution-time calculation, often leading to unsuccessful validation of the trajectories and was therefore not used in the final version and remained a theoretical implementation validated only in simulation.

The resulting equation system is solved, sampled at the control frequency of 1000 Hz, and validated against the physical joint limits. If the limits are violated, T is increased and the trajectory is recomputed. Once successfully validated, the complete trajectory is sent to the controller as a single command. The execution state machine verifies that the commanded robot motion actually begins, retries commands when necessary and waits for trajectory-completion feedback before progressing to the next phase. This prevents a stale completion message or lost command from causing the next stage to execute prematurely.

IV. RESULTS AND DISCUSSION

The implemented modules were integrated into a common ROS 2 workflow from sensor calibration to needle trajectory generation. Table I summarizes representative intermediate results obtained from the stored experimental dataset.

The localized target was estimated at

$${}^B\mathbf{p}_t = [0.2357 \quad -0.2408 \quad 0.0750]^T \text{ m}. \quad (24)$$

The selected needle-entry point was

$${}^B\mathbf{p}_e = [0.2590 \quad -0.3605 \quad 0.1776]^T \text{ m}, \quad (25)$$

producing an insertion distance of approximately 159.4 mm. A total of 286 candidates satisfied the configured geometric constraints, and the ten highest-ranked candidates were retained for possible downstream evaluation.

The results demonstrate the main advantage of the model-based approach. The depth sensor is not required to resolve the semantic target directly. Instead, the measured point cloud establishes the physical pose of the skeleton, while the reference STL provides the high-resolution geometric and target

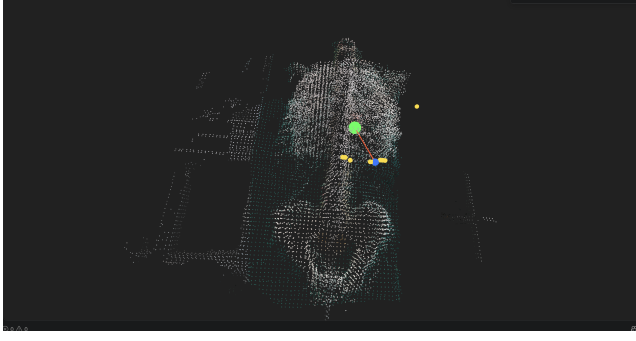


Fig. 4. Localized target and geometrically feasible needle-entry candidates in the robot base frame.

information. This separation is useful when the desired target is small, partially occluded, or poorly represented in depth data.

The modular architecture also provides a useful separation between anatomical and robotic constraints. The mapping system produces several collision-aware geometric insertion candidates, while the downstream kinematic system evaluates manipulator-specific feasibility. Consequently, an insertion candidate does not need to be regenerated merely because one robot configuration is unsuitable.

Several limitations remain. Registration currently depends on sufficient overlap between the reconstructed surface and reference STL, and ICP remains sensitive to poor initial alignment and large structural ambiguities. The lightweight robot collision checks do not replace a complete geometric collision model of the manipulator and surrounding environment. Most importantly, the stored results validate intermediate geometric consistency but do not provide an independent ground-truth measurement of final needle-tip placement accuracy. Therefore, the current system should be interpreted as a functional autonomous insertion prototype rather than a clinically validated system.

Future evaluation should measure the complete error chain from camera and hand-eye calibration through model registration, target localization, robot positioning, and final needle-tip placement.

The ROSBridge layer introduced a reliability problem for this workflow: it is not a trustworthy source for the timing and completion signals the state machine depends on. In practice, messages such as target-acceptance and capture-finished updates can be delayed, dropped, or arrive in a stale order, so the controller can miss state transitions. Because the bridge does not guarantee consistent delivery of those high-level events, the robot could receive a new target before the previous motion had actually finished, or the system could fail to register that a pose had been reached. In this case, the robots internal ROS1-based control would stop motion and require a restart. To compensate, the state machine had to be implemented locally in the scanning node, using its own motion/arrival checks rather than trusting the ROSBridge to deliver the exact lifecycle messages. This made the system

more robust, but it also meant the bridge was effectively acting as a weak link that forced the application logic to become explicit and defensive.

V. CONCLUSION

An autonomous robot-assisted needle-insertion pipeline was developed using a Franka Emika Panda manipulator, an eye-in-hand Azure Kinect, and ROS 2. The system integrates camera and hand-eye calibration, multi-view reconstruction, model registration, target and entry-point planning, inverse kinematics, and joint-trajectory generation within a common perception-to-action workflow. The use of a high-resolution reference model enables reliable target transfer to the physical scene, while multiple candidate insertion paths allow geometric and robot-specific feasibility to be considered separately. Future work should focus on quantitative end-to-end needle-placement accuracy, improved collision modelling, and feedback during physical insertion.

REFERENCES

- [1] Z. Zhang, "A flexible new technique for camera calibration," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000, doi: 10.1109/34.888718.
- [2] P. J. Besl and N. D. McKay, "A method for registration of 3-D shapes," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 14, no. 2, pp. 239–256, 1992, doi: 10.1109/34.121791.
- [3] K. S. Arun, T. S. Huang, and S. D. Blostein, "Least-squares fitting of two 3-D point sets," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-9, no. 5, pp. 698–700, 1987, doi: 10.1109/TPAMI.1987.4767965.
- [4] C. W. Wampler, "Manipulator inverse kinematic solutions based on vector formulations and damped least-squares methods," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 16, no. 1, pp. 93–101, 1986, doi: 10.1109/TSMC.1986.289285.
- [5] M. Tölgyessy, M. Dekan, L. Chovanec, and P. Hubinský, "Evaluation of the Azure Kinect and its comparison to Kinect V1 and Kinect V2," *Sensors*, vol. 21, no. 2, Art. no. 413, 2021, doi: 10.3390/s21020413.
- [6] G. Bocchicchio, "Controlling the KINOVA 7DOF," GitHub repository. [Online]. Available: https://github.com/Bocchicchio3/Controlling_the_KINOVA7DOF
- [7] L. Biagiotti and C. Melchiorri *Trajectory Planning. In: Trajectory Planning for Automatic Machines and Robots*. Springer, Berlin, Heidelberg, 2008.
- [8] RNM Project Description, Internal Report, 2024.